VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

AAT REPORT

ON

Object Oriented Java Programming(23CS3PCOOJ)

Submitted by

Sarthak Thrishuleshwar (1BM24CS260)

Sanjith C Tarigopula(1BM24CS257)

Saquib S Nadaf(1BM24CS259)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Sep-2025 to Jan-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the AAT work entitled "Object Oriented Java Programming(23CS3PCOOJ)" carried out by **Sarthak Thrishuleshwar (1BM24CS260), Sanjith Tarigopula(1BM24CS260), Saquib Nadaf(1BM24CS260),** who is bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025. The Lab report has been approved as it satisfies the academic requirements in respect of a Object oriented Java Programming (23CS3PCOOJ) work prescribed for the said degree.

Faculty: Dr Shubha V Rao Associate HOD Department of CSE, BMSCE	Dr Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index Sheet

Serial No.	TITLE	PAGE NO.
1	Problem Statement	1
2	Introduction	2
3	Overview of the project(Block Diagram)	3
4	Tools Used	4
5	OOJ Concepts Used	6
6	Implementation/code	9
7	Results	20
8	References	26

Problem Statement

In traditional voting systems, the voting process often involves manual procedures that are time-consuming, error-prone, and require significant human supervision. There is also a risk of multiple voting, inaccurate vote counting, and delayed result declaration. To overcome these limitations, there is a need for a simple computerized system that can conduct voting efficiently, securely, and accurately.

The problem is to design and implement an Online Voting System using Java that allows a predefined set of voters to cast their votes for a predefined list of candidates. The system must ensure that each voter is allowed to vote only once, thereby preventing duplicate voting. The voting process should validate voter identity, accept a valid candidate selection, and update the vote count correctly.

The system should maintain candidate details such as candidate ID, name, and vote count, and voter details such as voter ID and voting status. It must also provide a facility to display the current election results at any time. To simulate real-world voting scenarios, the system should be capable of handling multiple voting requests and should prevent race conditions during vote counting.

Additionally, the system should handle invalid operations gracefully, such as attempts to vote multiple times or selecting invalid candidates, using user-defined exception handling. To improve realism and performance, the voting operation should be implemented using multithreading, allowing multiple voters to cast votes concurrently while maintaining data consistency.

Objective of the Problem

The objective of this problem is to develop a secure, efficient, and modular online voting system that demonstrates the application of Object-Oriented Programming concepts, exception handling, multithreading, and Java collections, while simulating a real-world voting process in a controlled environment.

INTRODUCTION

With the rapid advancement of information technology, many traditional manual systems are being replaced by computerized solutions to improve efficiency, accuracy, and reliability. One such critical application is the voting system. Conventional voting methods involve extensive paperwork, manual verification, and physical presence of voters, which can lead to delays, human errors, and security concerns such as duplicate voting or incorrect vote counting.

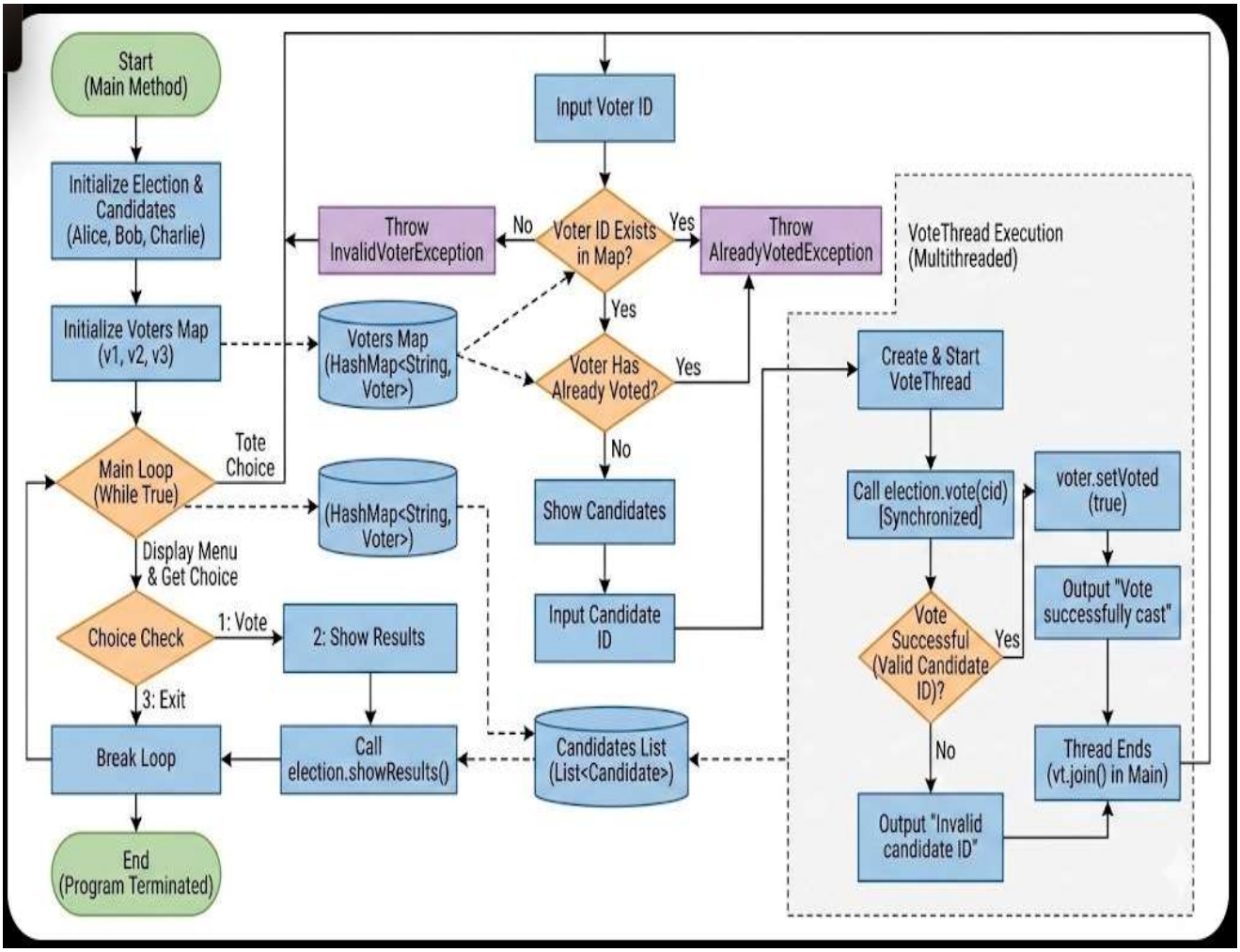
The Online Voting System is a Java-based application developed to simulate a simple, secure, and efficient voting process in a controlled digital environment. The system allows registered voters to cast their votes for predefined candidates while ensuring that each voter is permitted to vote only once. By automating the voting process, the system eliminates common problems associated with manual voting, such as vote duplication, incorrect tallying, and delayed result declaration.

This program is designed using Object-Oriented Programming (OOP) principles, which provide modularity, reusability, and maintainability. The application is divided into multiple classes such as Voter, Candidate, Election, and Main, each responsible for handling specific functionalities of the system. The use of Java collections like ArrayList and HashMap enables efficient storage and management of voter and candidate data.

To enhance system reliability and simulate real-world scenarios, the program incorporates user-defined exception handling to manage invalid operations such as multiple voting attempts. Additionally, multithreading is implemented to allow concurrent voting requests, ensuring that vote processing remains accurate even when multiple users attempt to vote simultaneously. Synchronization mechanisms are used to maintain data consistency during concurrent access.

Overall, this Online Voting System provides a clear demonstration of how Java can be used to design secure and efficient applications for managing voting processes. Although the system is intended for educational and demonstration purposes, it lays a strong foundation for understanding how real-world online voting systems can be developed using advanced programming concepts.

BLOCK DIAGRAM



This flowchart represents the working of a multithreaded online voting system. The program starts by initializing the election, candidates, and a voters map. The system continuously displays a menu allowing the user to vote, view results, or exit. When a voter chooses to vote, the system verifies whether the voter ID exists and checks if the voter has already voted, throwing appropriate exceptions for invalid or duplicate voting attempts. If valid, the voter selects a candidate, and a separate voting thread is created to safely record the vote using synchronized methods. Successful voting is confirmed, while invalid candidate selections are rejected. The admin or user can view results at any time, and the program terminates cleanly when the exit option is chosen.

TOOLS USED

The Online Voting System is developed using various software tools and programming utilities that support program development, execution, and testing. Each tool plays an important role in building a reliable and efficient application.

1. Java Programming Language

Java is the primary programming language used to develop the application. It provides object-oriented features such as classes, inheritance, encapsulation, and polymorphism, which are extensively used throughout the controllers, models, and repositories.

2. Spring Boot Framework

Spring Boot is used to build and run the web application efficiently. It simplifies configuration, enables rapid development, and provides built-in support for dependency injection, MVC architecture, and embedded servers.

3. Spring MVC

Spring MVC is used to handle HTTP requests and responses.
Annotations such as `@Controller`, `@RequestMapping`, `@GetMapping`, and `@PostMapping` are used to map URLs to controller methods and manage web navigation.

4. Spring Data JPA

Spring Data JPA is used for database interaction.
Repositories like `AdminRepository`, `VoterRepository`, and `CandidateRepository` provide CRUD operations without writing SQL queries manually.

5. Hibernate (JPA Implementation)

Hibernate acts as the ORM (Object Relational Mapping) tool.
It maps Java objects such as `Voter`, `Admin`, and `Candidate` to database tables and manages data persistence automatically.

6. BCrypt Password Encoder

`BCryptPasswordEncoder` is used for password encryption and security.
It ensures that admin and voter passwords are stored in encrypted form, preventing unauthorized access and improving application security.

7. HTTP Session Management

`HttpSession` is used to manage user sessions.
It helps track logged-in admins and voters, ensures authorization, and prevents unauthorized access to restricted pages.

8. Thymeleaf Template Engine

Thymeleaf is used for server-side rendering of HTML pages.
It dynamically displays data such as candidate lists, voting messages, results, and errors on web pages.

9. Embedded Tomcat Server

Spring Boot uses an embedded Apache Tomcat server to run the application.
This removes the need for external server configuration and allows the application to run directly.

10. REST and HTTP Protocol

The application uses HTTP methods such as GET and POST to process requests.
`ResponseEntity` is used to send HTTP responses, including exporting voter details as a text file.

11. Database Management System (MySQL / H2)

A relational database is used to store admin, voter, and candidate details.
Spring Boot supports databases like MySQL or H2, which can be configured based on deployment requirements.

12. Maven Build Tool

Maven is used for dependency management and project build automation.
It manages libraries such as Spring Boot, Spring Security, JPA, and Thymeleaf.

OOJ CONCEPTS USED

The Online Voting System is designed using object-oriented programming principles, which help in organizing the program into reusable, manageable, and real-world entities. The following OOP concepts are used in the program:

1. Class and Object

The most fundamental object-oriented concept used in the code is Class and Object.
Classes such as AdminController, VoterController, Admin, Voter, and Candidate represent blueprints. Objects of these classes are created and managed by the Spring framework at runtime.

For example:

- Admin, Voter, and Candidate are model classes representing real-world entities.
- Repositories like AdminRepository, VoterRepository, and CandidateRepository manage objects of these classes.

Thus, real-world entities such as voters, admins, and candidates are modeled using objects.

2. Encapsulation

Encapsulation is achieved by binding data and methods together and restricting direct access to the internal state.

In the given code:

- Fields such as passwords, vote counts, and voting status are kept private inside model classes.
- Access is provided through getter and setter methods like getVotes(), setHasVoted(true), and getPassword().

This ensures that sensitive data such as passwords and vote counts cannot be modified directly, improving security and data integrity.

3. Abstraction

Abstraction is implemented by hiding implementation details and exposing only necessary functionality.

Examples from the code:

- Repository interfaces (AdminRepository, VoterRepository, CandidateRepository) hide database operations.
- The controller classes interact with repositories using simple methods like save(), findAll(), and findById() without knowing how data is stored internally.

This simplifies development and makes the system easier to maintain.

4. Inheritance

Inheritance is indirectly used through the Spring Framework.

- Controller classes inherit behavior from Spring's base classes and annotations such as @Controller.
- Repository interfaces extend Spring Data JPA interfaces like JpaRepository.

This allows the application to reuse existing functionality such as CRUD operations and HTTP request handling without rewriting code.

5. Polymorphism

Polymorphism is used when the same method behaves differently based on context.

Examples include:

- The findById() method works for different entities (Admin, Voter, Candidate) but behaves according to the object type.
- Spring handles method overloading in controllers using annotations like @GetMapping and @PostMapping for the same URL path with different HTTP methods.

This improves flexibility and code reuse.

6. Dependency Injection

Dependency Injection is a core OOP principle heavily used in the code through Spring.

Examples:

- @Autowired injects dependencies such as repositories and password encoders into controllers.
- Objects are not created manually using new, but are provided by the Spring container.

This promotes loose coupling, easier testing, and better scalability.

7. Modularity

The system is modularized into different packages:

- controller handles request processing
- model represents data entities
- repository manages database access

Each module has a single responsibility, which follows object-oriented design principles and improves maintainability.

8. Reusability

Reusability is achieved through:

- Common repository methods (save, findAll, count)
- Reusable model classes like Voter and Candidate
- Shared logic such as vote counting and winner determination

This reduces code duplication and improves efficiency.

9. Association Between Objects

The code demonstrates association between objects:

- A Voter is associated with the voting process.
- A Candidate is associated with vote counting.
- Controllers coordinate interactions between these objects.

These associations model real-world relationships within the voting system.

10. Separation of Concerns

Each class has a clearly defined role:

- Controllers manage HTTP requests and responses
- Models represent application data
- Repositories handle persistence

This separation aligns with object-oriented best practices and improves clarity and extensibility.

IMPLEMENTATION / CODE

AdminConroller.java

```
package com.voting.votingsystem.controller;


import java.io.FileWriter;


import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.http.ResponseEntity;

import org.springframework.stereotype.Controller;

import org.springframework.ui.Model;

import org.springframework.web.bind.annotation.*;

import java.io.File;

import java.io.FileWriter;


import com.voting.votingsystem.model.Admin;

import com.voting.votingsystem.model.Candidate;

import com.voting.votingsystem.model.Voter;

import com.voting.votingsystem.repository.AdminRepository;

import com.voting.votingsystem.repository.CandidateRepository;

import com.voting.votingsystem.repository.VoterRepository;

import com.voting.votingsystem.model.Voter;

import org.springframework.http.ResponseEntity;

import com.voting.votingsystem.model.Voter;


import jakarta.servlet.http.HttpSession;


import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;


@Controller
```

```
@RequestMapping("/admin")
public class AdminController {

    @Autowired
    private CandidateRepository candidateRepo;

    @Autowired
    private AdminRepository adminRepo;

    @Autowired
    private BCryptPasswordEncoder encoder;

    //login

    @GetMapping("/login")
    public String adminLogin() {
        return "admin-login";
    }

    @PostMapping("/login")
    public String processAdminLogin(@RequestParam String username,
            @RequestParam String password,
            HttpSession session,
            Model model) {

        Admin admin = adminRepo.findById(username).orElse(null);

        if (admin == null || !encoder.matches(password, admin.getPassword())) {
            model.addAttribute("error", "Invalid admin credentials");
```

```
        return "admin-login";
    }

    session.setAttribute("admin", username);
    return "redirect:/admin/dashboard";
}

//dashboard
@Autowired
private VoterRepository voterRepo;

@GetMapping("/dashboard")
public String dashboard(HttpSession session, Model model) {

    if (session.getAttribute("admin") == null) {
        return "redirect:/admin/login";
    }

    model.addAttribute("candidates", candidateRepo.findAll());
    return "admin-dashboard";
}

@PostMapping("/addVoters")
public String addVoters(@RequestParam int count,
        HttpSession session,
        Model model) {

    if (session.getAttribute("admin") == null) {
        return "redirect:/admin/login";
```

```
    }

    //reset
    voterRepo.deleteAll();



    for (int i = 1; i <= count; i++) {
        String voterId = "v" + i;
        String rawPassword = voterId + "123"; // demo rule

        voterRepo.save(new Voter(
                voterId,
                encoder.encode(rawPassword)));
    }

    model.addAttribute("message", count + " voters created successfully");
    return "redirect:/admin/dashboard";
}

//addcan

@PostMapping("/addCandidate")
public String addCandidate(@RequestParam String name,
        HttpSession session) {

    if (session.getAttribute("admin") == null) {
        return "redirect:/admin/login";
    }
```

```
        candidateRepo.save(new Candidate(name));
        return "redirect:/admin/dashboard";
    }

    //results

    @GetMapping("/results")
public String results(HttpSession session, Model model) {

    if (session.getAttribute("admin") == null) {
        return "redirect:/admin/login";
    }

    var candidates = candidateRepo.findAll();
    model.addAttribute("candidates", candidates);

    if (!candidates.isEmpty()) {
        Candidate winner = candidates.stream()
                .max((c1, c2) -> Integer.compare(c1.getVotes(), c2.getVotes()))
                .orElse(null);

        model.addAttribute("winner", winner);
    }

    return "admin-results";
}

    private boolean isVotingEnded() {
    return voterRepo.count() == voterRepo.findAll()
```

```
            .stream()
            .filter(Voter::isHasVoted)
            .count();
}

private Candidate getWinner() {
    return candidateRepo.findAll()
            .stream()
            .max((c1, c2) -> Integer.compare(c1.getVotes(), c2.getVotes()))
            .orElse(null);
}
    //logout

    @GetMapping("/logout")
    public String logout(HttpSession session) {
        session.invalidate();
        return "redirect:/admin/login";
    }
    //@PostMapping("/exportVoters")
@GetMapping("/exportVoters")
public ResponseEntity<String> exportVoters(HttpSession session) {

    if (session.getAttribute("admin") == null) {
        return ResponseEntity.status(403).body("Unauthorized");
    }

    StringBuilder content = new StringBuilder();
    content.append("VOTER ID : PASSWORD\n");
    content.append("------------------\n");
```

```
    for (Voter v : voterRepo.findAll()) {
        String rawPassword = v.getVoterId() + "123";
        content.append(v.getVoterId())
                .append(" : ")
                .append(rawPassword)
                .append("\n");
    }
    return ResponseEntity.ok()
            .header("Content-Disposition", "inline; filename=voters.txt")
            .header("Content-Type", "text/plain")
            .body(content.toString());
}
}
```

VoteController.java

```
package com.voting.votingsystem.controller;


import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import jakarta.servlet.http.HttpSession;


import com.voting.votingsystem.model.Candidate;
import com.voting.votingsystem.model.Voter;
import com.voting.votingsystem.repository.CandidateRepository;
import com.voting.votingsystem.repository.VoterRepository;
```

```
@Controller
@RequestMapping("/voter")
public class VoterController {
    @Autowired
    private CandidateRepository candidateRepo;

    @Autowired
    private VoterRepository voterRepo;

    @GetMapping("/login")
    public String loginPage() {
        return "voter-login";
    }

    @GetMapping("/candidates")
public String showCandidates(HttpSession session, Model model) {

    String voterId = (String) session.getAttribute("voterId");

    if (voterId == null) {
        return "redirect:/voter/login";
    }

    long votedCount = voterRepo.findAll()
            .stream()
            .filter(Voter::isHasVoted)
```

```
            .count();

    if (votedCount == voterRepo.count()) {
        model.addAttribute("message", "Voting has ended");
        return "vote-ended";
    }

    model.addAttribute("voterId", voterId);
    model.addAttribute("candidates", candidateRepo.findAll());
    return "voter-candidates";
}

    @Autowired
private BCryptPasswordEncoder encoder;

    @PostMapping("/login")
    public String processLogin(@RequestParam String voterId,
            @RequestParam String password,
            HttpSession session,
            Model model) {

        Voter voter = voterRepo.findById(voterId).orElse(null);

        if (voter == null || !encoder.matches(password, voter.getPassword())
) {
            model.addAttribute("error", "Invalid Voter ID or Password");
            return "voter-login";
        }
```

```
        if (voter.isHasVoted()) {
            model.addAttribute("error", "You have already voted");
            return "voter-login";
        }

        session.setAttribute("voterId", voterId);
        return "redirect:/voter/candidates";
    }
    @PostMapping("/vote")
public String vote(@RequestParam int candidateId,
                   HttpSession session,
                   Model model) {

    String voterId = (String) session.getAttribute("voterId");

    if (voterId == null) {
        return "redirect:/voter/login";
    }

    Voter voter = voterRepo.findById(voterId).orElse(null);
    Candidate candidate = candidateRepo.findById(candidateId).orElse(null);

    if (voter == null || candidate == null || voter.isHasVoted()) {
        model.addAttribute("message", "Vote failed");
        return "vote-ended";
    }

    //  Cast vote
    candidate.incrementVote();
```

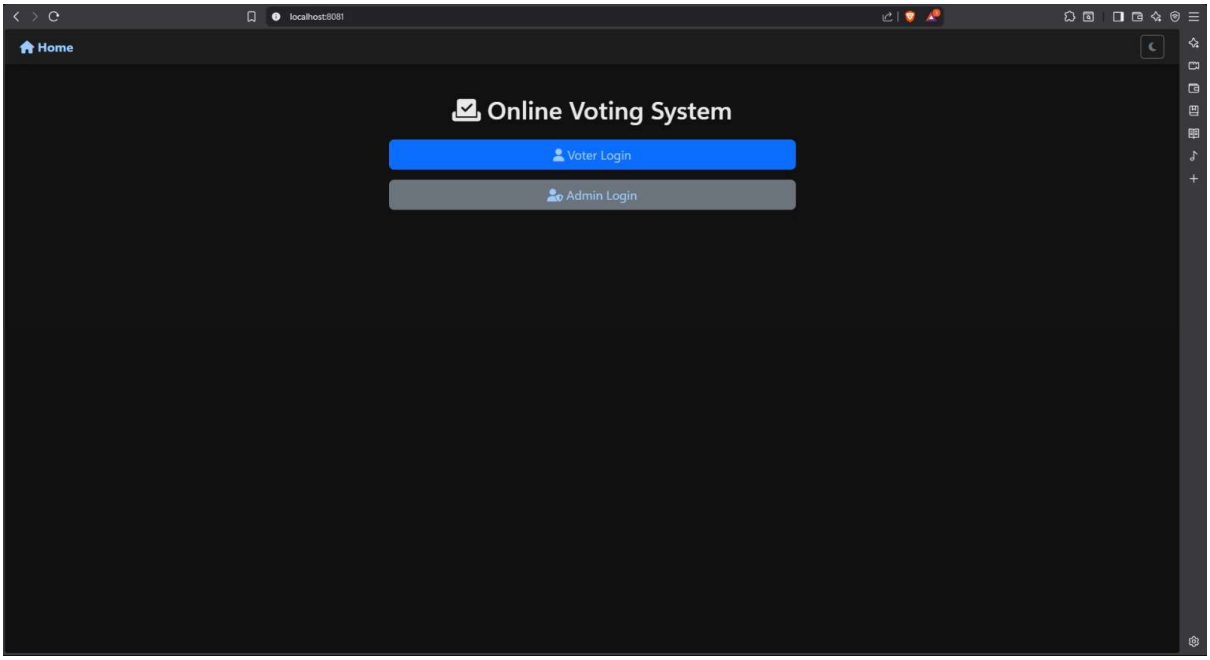
```
    voter.setHasVoted(true);

    candidateRepo.save(candidate);
    voterRepo.save(voter);

    //  End session for this voter
    session.invalidate();

    // CHECK IF THIS WAS THE LAST VOTE
    long totalVoters = voterRepo.count();
    long votedCount = voterRepo.findAll()
            .stream()
            .filter(Voter::isHasVoted)
            .count();

    if (votedCount == totalVoters) {
        //  Find winner
        Candidate winner = candidateRepo.findAll()
                .stream()
                .max((c1, c2) -> Integer.compare(c1.getVotes(), c2.getVotes()))
                .orElse(null);
        model.addAttribute("winner", winner);
        return "final-result";   //  AUTO RESULT POPUP
    }
    // Normal flow for non-last voters
    model.addAttribute("message", "Vote added successfully");
    return "vote-result";
}
}
```

RESULTS/OUTPUT

1) Home Page – Online Voting System

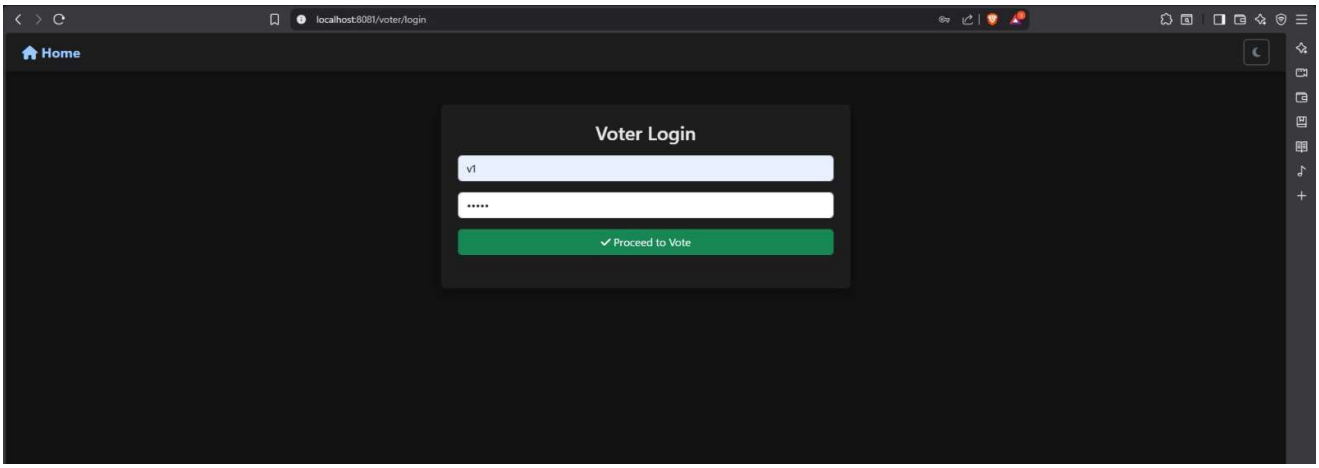
This is the main entry page of the system. It provides two options:

- Voter Login for registered voters to cast their vote.
- Admin Login for the administrator to manage candidates, voters, and view results.
  This separation ensures role-based access and security.



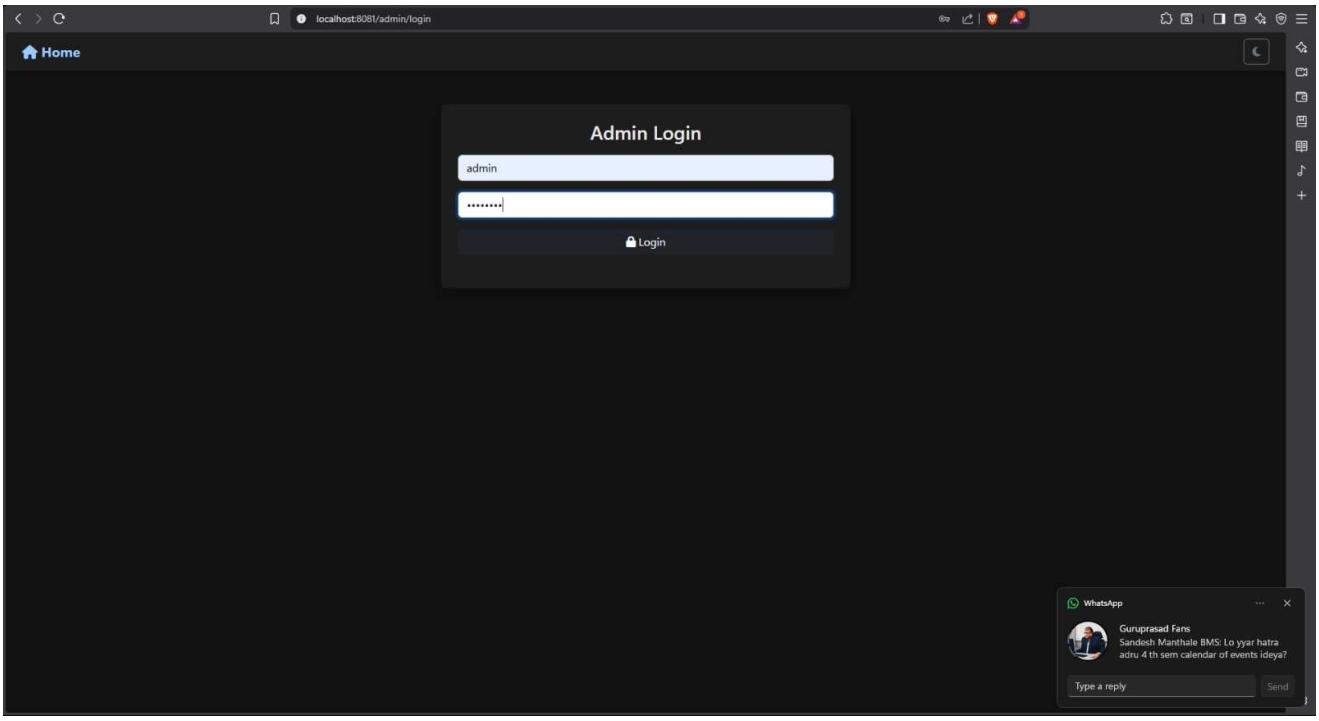
2) Voter Login Page

This screen allows a voter to authenticate using their voter ID and password.
Only valid voters can proceed further, ensuring that unauthorized users cannot vote.



3) Admin Login Page

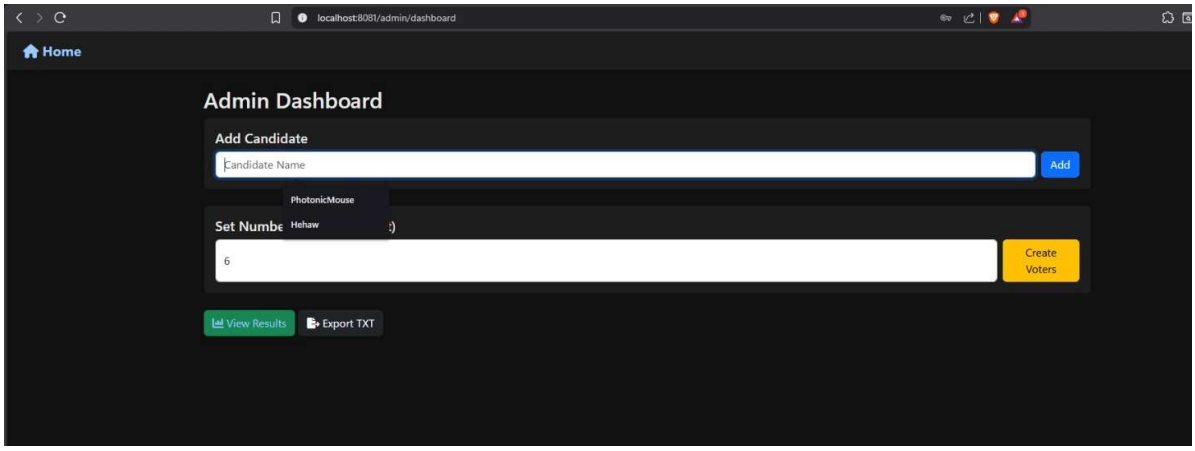
The admin login page authenticates the administrator.
After successful login, the admin can add candidates, create voters, and manage the election process.



4) Admin Dashboard

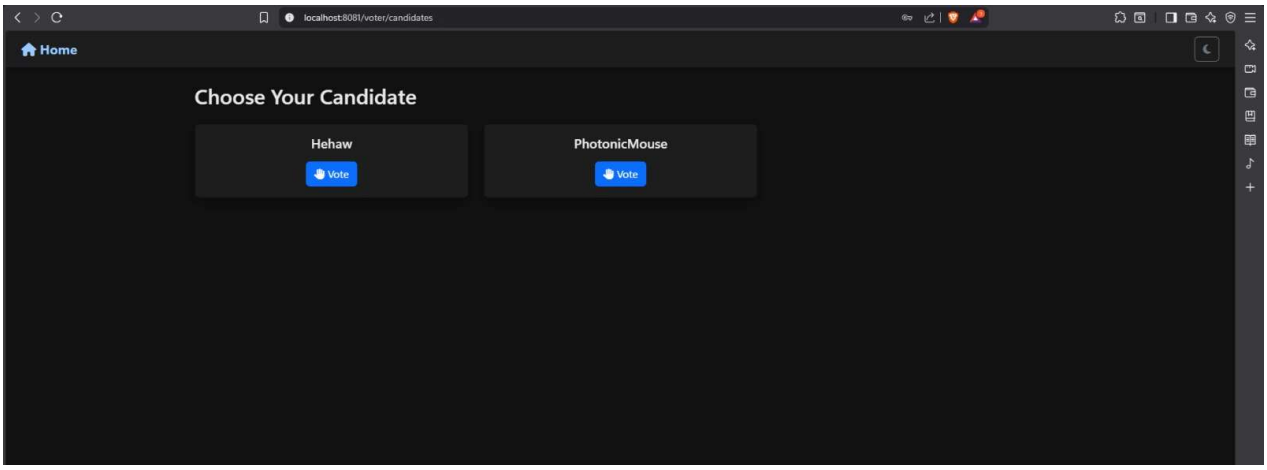
This dashboard enables the administrator to:

- Add new candidates
- Set the number of voters
- View election results
- Export results for records
  It provides complete control and monitoring of the voting system.



5) Candidate Selection Page (Voter Side)

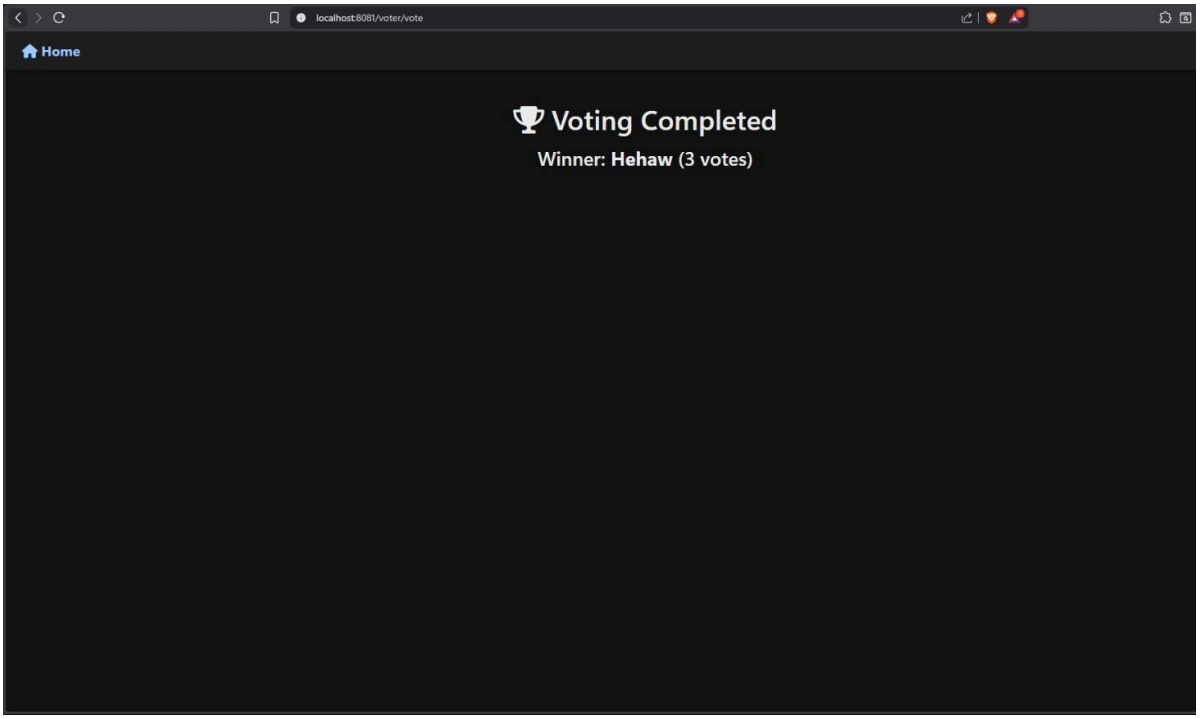
Once logged in, the voter is shown a list of available candidates**.**
Each candidate has a Vote button, allowing the voter to cast exactly one vote, ensuring fairness.



6) Voting Completion / Result Page

This final screen confirms that the voting process is complete and displays:

- The winner's name
- The total number of votes received
  It ensures transparency by clearly announcing the election outcome.



REFERENCES

- **Herbert Schildt**, *Java: The Complete Reference*, McGraw Hill Education, 11th Edition.
– Referred for core Java concepts, OOP principles, exception handling, and multithreading.

- **Oracle Corporation**, *Java Platform, Standard Edition Documentation*.
Available at: https://docs.oracle.com/javase/
– Used for understanding Java syntax, Scanner class, Collections Framework, and threading.

- **Kathy Sierra and Bert Bates**, *Head First Java*, O'Reilly Media.
– Referred for object-oriented programming concepts and practical examples.

- **Oracle Java Tutorials**, *Object-Oriented Programming Concepts*.
Available at: https://docs.oracle.com/javase/tutorial/java/concepts/
– Used for understanding encapsulation, inheritance, polymorphism, and abstraction.

- **GeeksforGeeks**, *Java Programming Tutorials*.
Available at: https://www.geeksforgeeks.org/java/
– Referred for examples on collections, exception handling, and multithreading.

- **W3Schools**, *Java Tutorial*.
Available at: https://www.w3schools.com/java/
– Used for quick reference to Java syntax and input/output handling.